15.03.2026

# Swisscom Network Observability

## Transforms and Innovates Network Operations

**Wanting Du** - wanting.du@swisscom.com
**Thomas Graf** – thomas.graf@swisscom.com

*Picture: Apollo 8, December 24th 1968*

swisscom

# Agentic AI and its Network Analytics Applicability

## Exampled on Network Anomaly Detection

Autonomous, Agentic AI, Generative AI and Large Language Models are recent buzzwords which excite beyond the network industry.

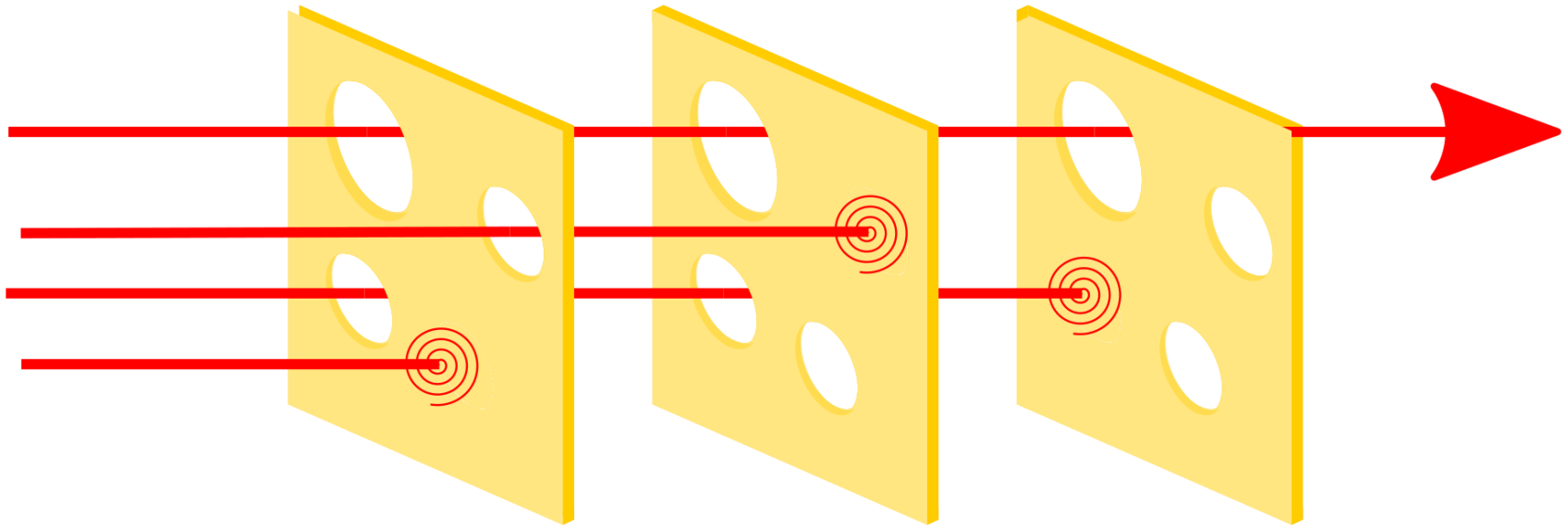
In order to explain how this affects Network Analytics, with Network Anomaly Detection as a concrete example, **we need to understand the foundation, how Data Models, Rich Semantics, Ontology, Taxonomy and Knowledge Graphs work**. Data models represent a basic form of organizing structured data. Semantics give data meaning. Taxonomy its relation.

*"AI Fascinates Throughout History"*

“ With postmortems we learn. *We can't have enough of them. Agentic AI can help us to automate postmortems by analyzing and gathering data and refining the incident detection.* “

# Swiss cheese model of accident causation

## Mostly multiple interacting causes lead to an incident



“ **The Swiss cheese model of accident causation is a model used in risk analysis and risk management**. It likens human systems to multiple slices of Swiss cheese, which have randomly placed and sized holes in each slice, stacked side by side, in which the risk of a threat becoming a reality is mitigated by the different types of defenses which are "layered" behind each other. Therefore, in theory, **lapses and weaknesses in one defense** (e.g. a hole in one slice of cheese) **do not allow a risk to materialize, since other defenses also exist** (e.g. other slices of cheese), **to prevent a single point of failure**. “

https://en.wikipedia.org/wiki/Swiss_cheese_model

# September 4th, Maximum Prefix BGP Peer State Change

## External Cloud Access, AWS Peering

**Cosmos Bright Lights monitoring 64498:46982 64498:55780 L3 VPN in real-time during incident window – [Pivot Link]**

**Long time ago, when a new L3 VPN connectivity service was created, the maximum prefix limit on a BGP peering at AWS public cloud was defined but not proactively monitored.**

**On September 4th at 08:55 additional prefixes were advertised** into the L3 VPN and observed by Cosmos Bright Lights Network Anomaly Detection but nor alerted.

**At 09:08 additional prefixes were advertised and soon after the BGP peering was teared.** Before peering teardown, the AWS BGP speaker notified Swisscom's PE node the reason of the peer down. The Swisscom PE node forwarded in the BMP peer_down notification to the data collection where the peer_down but not the reason was decoded. **Cosmos Bright Lights Network Anomaly Detection observed the topology and peering state changes but did not alert.**

**Some time later network operation was contacted due to customer facing incident…**

# YANG Data Taxonomy

## oper-status and last-changed in ietf-interfaces

For instance take ietf-interfaces.yang as an example for an interface data model and the leaf "**oper-status**" defines the operational state of that interface and the leaf "**last-change**" when the last state change occurred. YANG schema trees facilitate a visual representation, the taxonomy of YANG data models.

```
module: ietf-interfaces
     +--rw interfaces
        +--rw interface* [name]
           +--rw name                          string
           +--rw description?                  string
           +--rw type                          identityref
           +--rw enabled?                      boolean
           +--rw link-up-down-trap-enable?     enumeration {if-mib}?
           +--ro admin-status                  enumeration {if-mib}?
           +--ro oper-status                   enumeration
           +--ro last-change?                  yang:date-and-time
           +--ro if-index                      int32 {if-mib}?
           +--ro phys-address?                 yang:phys-address
           +--ro higher-layer-if*              interface-ref
           +--ro lower-layer-if*               interface-ref
           +--ro speed?                        yang:gauge64
           +--ro statistics
              +--ro discontinuity-time     yang:date-and-time
              +--ro in-octets?             yang:counter64
              +--ro in-unicast-pkts?       yang:counter64
              +--ro in-broadcast-pkts?     yang:counter64
              +--ro in-multicast-pkts?     yang:counter64
              +--ro in-discards?           yang:counter32
              +--ro in-errors?             yang:counter32
              +--ro in-unknown-protos?     yang:counter32
              +--ro out-octets?            yang:counter64
              +--ro out-unicast-pkts?      yang:counter64
              +--ro out-broadcast-pkts?    yang:counter64
              +--ro out-multicast-pkts?    yang:counter64
              +--ro out-discards?          yang:counter32
              +--ro out-errors?            yang:counter32
```

# YANG Semantics

## oper-status in ietf-interfaces

Looking at "oper-status" in the data model definition we see "**description**" and "**reference**" as properties, together with identities, they describe the meaning of the data or in other words the semantics in YANG.

```
leaf oper-status {
            type enumeration {
              enum up {
                value 1;
                description
                  "Ready to pass packets.";
              }
              enum down {
                value 2;
                description
                  "The interface does not pass any packets.";
              }
              enum testing {
                value 3;
                description
                  "In some test mode.  No operational packets can
                   be passed.";
              }
              enum unknown {
                value 4;
                description
                  "Status cannot be determined for some reason.";
              }
              enum dormant {
                value 5;
                description
                  "Waiting for some external event.";
              }
              enum not-present {
                value 6;
                description
                  "Some component (typically hardware) is missing.";
              }
              enum lower-layer-down {
                value 7;
                description
                  "Down due to state of lower-layer interface(s).";
              }
            }
            config false;
            mandatory true;
            description
              "The current operational state of the interface.

               This leaf has the same semantics as ifOperStatus.";
            reference
              "RFC 2863: The Interfaces Group MIB - ifOperStatus";
          }
```

# September 4th, Maximum Prefix BGP Peer State Change

## BGP notifications and BMP Peer Down Message

RFC 7854 - BGP Monitoring Pro

datatracker.ietf.org/doc/

RFC 7854 **Proposed Standard**

**4.9. Peer Down Notification**

This message is used to indicate that a peering session was terminated.

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+
|    Reason     |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|            Data (present if Reason = 1, 2 or 3)               |
~                                                               ~
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

Reason indicates why the session was closed. Defined values are:

- o Reason 1: The local system closed the session. Following the Reason is a BGP PDU containing a BGP NOTIFICATION message that would have been sent to the peer.
- o Reason 2: The local system closed the session. No notification message was sent. Following the reason code is a 2-byte field containing the code corresponding to the Finite State Machine (FSM) Event that caused the system to close the session (see Section 8.1 of [RFC4271]). Two bytes both set to 0 are used to indicate that no relevant Event code is defined.
- o Reason 3: The remote system closed the session with a notification message. Following the Reason is a BGP PDU containing the BGP NOTIFICATION message as received from the peer.
- o Reason 4: The remote system closed the session without a notification message. This includes any unexpected termination of the transport session, so in some cases both the local and remote systems might consider this to apply.
- o Reason 5: Information for this peer will no longer be sent to the monitoring station for configuration reasons. This does not, strictly speaking, indicate that the peer has gone down, but it does indicate that the monitoring station will not receive updates for the peer.

RFC 4486: Subcodes for BGP Ce

www.rfc 130%

**3. Subcode Definition**

The following subcodes are defined for the Cease NOTIFICATION message:

| Subcode | Symbolic Name |
|---|---|
| 1 | Maximum Number of Prefixes Reached |
| 2 | Administrative Shutdown |
| 3 | Peer De-configured |
| 4 | Administrative Reset |
| 5 | Connection Rejected |
| 6 | Other Configuration Change |
| 7 | Connection Collision Resolution |
| 8 | Out of Resources |

NetGauze/crates/bgp-pkt at ma

github.com/NetGauze/Ne

Files main NetGauze / crates / bgp-pkt / Top

### Supported BGP Protocol features

#### Supported message types

| Message Type | RFCs | notes |
|---|---|---|
| Open | RFC 4271 | See below for the supported capabilities |
| Update | RFC 4271 | See below for the supported path attributes |
| Notification | RFC 4271 | See below for the supported notif sub-codes |
| KeepAlive | RFC 4271 | |
| RouteRefresh | RFC 2918 and RFC 7313 | See below for the supported Route refresh ops |

**BGP notifications propagate reason for BGP peer tear down. BMP mirrors BGP notification at BGP Adj-RIB In with reason code 3. If no notification was received but TCP reset was received, then reason code 4 is used.**

Show BGP neighbor output in CLI

```
RP/0/RP0/CPU0:pca01ro1072olt#sho bgp vrf EXTCC-00508-00
neighbor 100.100.98.209
.......
Time since last notification received from neighbor:
00:34:13
  Error Code: maximum number of prefixes reached
  Notification data received:
```

# September 4th, Maximum Prefix BGP Peer State Change

## draft-ietf-nmop-network-anomaly-semantics - Symptoms: **Action – Reason – Trigger**

draft-ietf-nmop-network-anomaly-semantics-03 None

| | | |
|---|---|---|
| Reachability | Withdraw | Stale |
| Reachability | Withdraw | Route Policy Filtered |
| Reachability | Withdraw | Maximum Number of Prefixes Reached |
| Adjacency | Established | Peer |
| Adjacency | Established | Link-Layer |
| Adjacency | Locally Teared Down | Peer |
| Adjacency | Remotely Teared Down | Peer |
| Adjacency | Locally Teared Down | Link-Layer |
| Adjacency | Remotely Teared Down | Link-Layer |
| Adjacency | Locally Teared Down | Administrative |
| Adjacency | Remotely Teared Down | Administrative |
| Adjacency | Locally Teared Down | Maximum Number of Prefixes Reached |
| Adjacency | Remotely Teared Down | Maximum Number of Prefixes Reached |
| Adjacency | Locally Teared Down | Transport Connection Failed |

https://datatracker.ietf.org/doc/html/draft-ietf-nmop-network-anomaly-semantics-03#symptom_control_plane_actions_table

draft-ietf-nmop-network-anomaly-semantics-03 None

**4.2.1. YANG Tree**

Figure 1 contains the YANG tree diagram [RFC8340] of the 'ietf-network-anomaly-symptom-cbl' module. It augments the 'ietf-relevant-state' module defined in [I-D.ietf-nmop-network-anomaly-lifecycle].

For each Symptom, the following parameters can be assigned: an Action, a Reason and a Trigger describing the Symptom; a concern score indicating how critical the Symptom is; and the associated network plane.

Where the season enumeration declares wherever a workday or a holiday hase been taken into consideration for Contextual Outliers. The template describes which approach and parameters have been used in the Service Disruption Detection as described in Section 3.2 of [I-D.ietf-nmop-network-anomaly-architecture]

```
module: ietf-network-anomaly-symptom-cbl

  augment /rsn:relevant-state/rsn:anomaly/rsn:symptom:
    +--rw action?          string
    +--rw reason?          string
    +--rw trigger?         string
    +--rw network-plane?   enumeration
    +--rw template?        string
    +--rw season?          enumeration
  augment /rsn:relevant-state-notification/rsn:anomaly/rsn:symptom:
    +-- action?          string
    +-- reason?          string
    +-- trigger?         string
    +-- network-plane?   enumeration
    +-- template?        string
    +-- season?          enumeration
```

https://datatracker.ietf.org/doc/html/draft-ietf-nmop-network-anomaly-semantics-03#section-4.2.1

In Postmortems, the causality is a key concern. With symptoms we describe the causality tree in a human and machine-readable ontology.

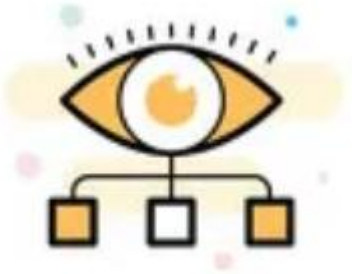
The symptoms is defined in semantic triple. Where "action" is the object, "trigger" the predicate and the reason "subject".

With semantic triple we knowledge graph.

# Problem Statement and Motivation

## How it is being addressed in which document

## Network Anomaly Detection



When operational or configurational changes in connectivity services are happening, **the objective is to detect interruption at network operation faster than the users using those connectivity services**.

In order to achieve this objective, **automation in network monitoring is required.** This automation needs to **monitor network changes holistically** by monitoring all 3 network planes simultaneously and detect whether that change is service disruptive.

Through network incidents postmortems we network operators **learn and improve so does network anomaly detection** and supervised and semi-supervised machine learning. **With more and more incidents the postmortem process demands automation** and with the standardization of labeled network incident collaboration among network operators, vendors and academia is facilitated.

- draft-ietf-nmop-network-anomaly-architecture describes the motivation and architecture and the relationship to other two documents.
- draft-ietf-nmop-network-anomaly-semantics defines Symptom semantics to enable standardized data exchange to validate results with network engineers and improve supervised and semi-supervised machine learning systems.
- draft-ietf-nmop-network-anomaly-lifecycle describes on managing the lifecycle process, in order to facilitate network engineers to interact with the network anomaly detection system to refine the detection abilities over time.

# IETF NMOP - Semantic Metadata Annotation for Network Anomaly Detection

## draft-ietf-nmop-network-anomaly-semantics – Schema Tree

```
notifications:
   +---n relevant-state-notification
      +--ro publisher
      |  +--ro id?        yang:uuid
      |  +--ro name       string
      |  +--ro version?   string
      +--ro id                                yang:uuid
      +--ro uri?                              inet:uri
      +--ro description?                      string
      +--ro start-time                        yang:date-and-time
      +--ro end-time?                         yang:date-and-time
      +--ro smcblsymptom:strategy?            string
      +--ro confidence-score?                 score
      +--ro concern-score                     score
      +--ro (service)?
      |  +--:(smtopology:l2vpn)
      |  |  +--ro smtopology:vpn-service* [vpn-id]
      |  |     +--ro smtopology:vpn-id                 string
      |  |     +--ro smtopology:uri?                   inet:uri
      |  |     +--ro smtopology:vpn-name?              string
      |  |     +--ro smtopology:site-ids*              string
      |  |     +--ro smtopology:change-id?             yang:uuid
      |  |     +--ro smtopology:change-start-time?
      |  |     |       yang:date-and-time
      |  |     +--ro smtopology:change-end-time?
      |  |             yang:date-and-time
      |  +--:(smtopology:l3vpn)
      |     +--ro smtopology:vpn-service* [vpn-id]
      |        +--ro smtopology:vpn-id                 string
      |        +--ro smtopology:uri?                   inet:uri
      |        +--ro smtopology:vpn-name?              string
      |        +--ro smtopology:site-ids*              string
      |        +--ro smtopology:change-id?             yang:uuid
      |        +--ro smtopology:change-start-time?
      |        |       yang:date-and-time
      |        +--ro smtopology:change-end-time?
      |                yang:date-and-time
```

```
notifications:
   +---n relevant-state-notification
   +--ro anomaly* [id revision]
         +--ro id                                  yang:uuid
         +--ro revision                            yang:counter32
         +--ro uri?                                inet:uri
         +--ro state                               identityref
         +--ro description?                        string
         +--ro start-time
         |       yang:date-and-time
         +--ro end-time?
         |       yang:date-and-time
         +--ro confidence-score?                   score
         +--ro pattern?                            identityref
         +--ro annotator
         |  +--ro id?                yang:uuid
         |  +--ro name               string
         |  +--ro version?           string
         |  +--ro annotator-type?    enumeration
         +--ro symptom!
         |  +--ro id                                yang:uuid
         |  +--ro concern-score                     score
         |  +--ro smcblsymptom:action?              string
         |  +--ro smcblsymptom:reason?              string
         |  +--ro smcblsymptom:trigger?             string
         |  +--ro smcblsymptom:network-plane?       enumeration
         |  +--ro smcblsymptom:template?            string
         |  +--ro smcblsymptom:season?              Enumeration
         +--ro smtopology:vpn-node-terminations*
                 [hostname route-distinguisher]
            +--ro smtopology:hostname                 inet:host
            +--ro smtopology:route-distinguisher      string
            +--ro smtopology:peer-ip*                 inet:ip-address
            +--ro smtopology:next-hop*                inet:ip-address
            +--ro smtopology:interface-id*            uint32
```

**Shows the observed symptoms, the network dimensions triggering and connectivity service impacted.**

# Anomaly Detection Refinement Lifecycle (1)

## Gaining **validated** knowledge

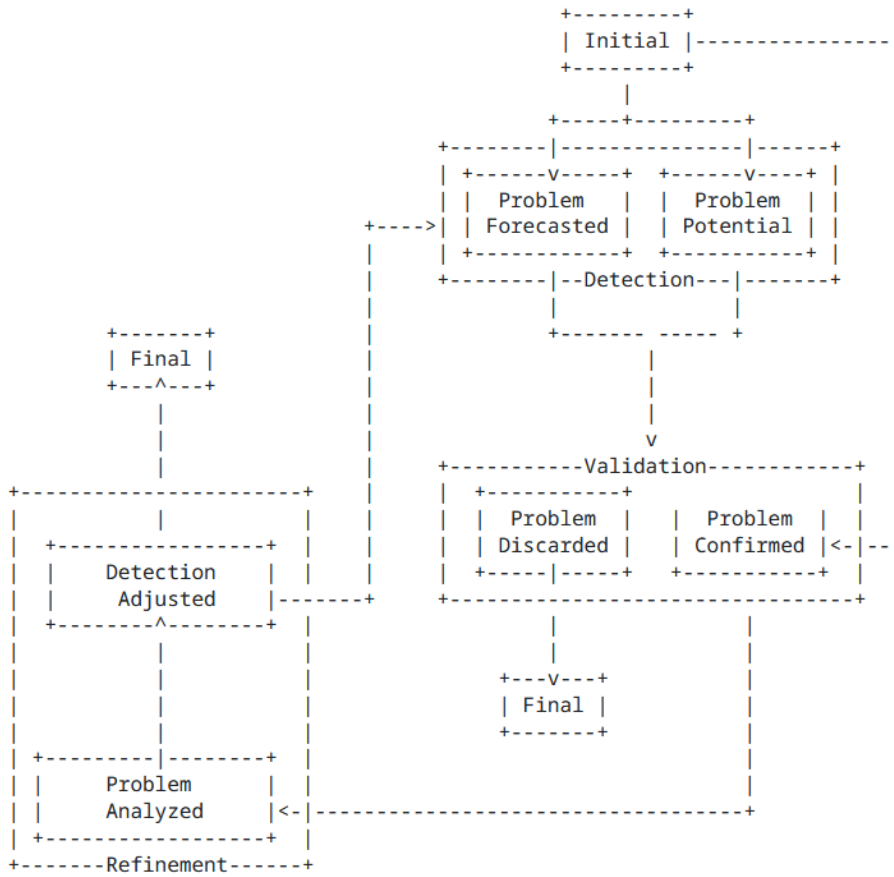
```
                        +-------------+
            +--------> |  Detection  | ---------+
Adjustments |           +-------------+          | Symptoms
            |                                    |
            |                                    v
    +------------+                       +------------+
    | Refinement |<--------------------- | Validation |
    +------------+        Problem        +------------+
                        Confirmation
```

With postmortems we network operators **gain insights into the reasoning of network incidents** by looking at what happened, which operational metrics were available and which analytical conclusion we derived. Obviously in this postmortem process we gather and link this knowledge. Obviously, **this linked knowledge can be graphed.** Obviously out of this linked and graphed knowledge **new knowledge can be derived with Large Language Models and with Agentic AI reasoning can be deducted** and autonomous action can be applied, like new and improved disruption checks are being proposed and tested as part of the network anomaly detection lifecycle workflow.

# Anomaly Detection Refinement Lifecycle (2)

## Gaining **validated** knowledge

**Data replay** is an important tool in Network Anomaly Detection. It **helps to refine the outlier detection**. To find the right hyper parameters for the right algorithm. With data replay we test and verify. **With Agentic AI we can expand to what-if**. Besides trying out different hyper parameters and algorithms we can now leverage based on Knowledge Graph relationships and Agentic AI inference new knowledge, new untapped aspects of modeled network data.

**Our Conclusion:**

Agentic AI can help to automate the refinement process of Network Anomaly Detection. **Would you agree?**